

Mohammed Aljomaily

Software Portfolio

Software Engineer | Backend, Cloud & Applied AI

Beylikdüzü, Istanbul, Turkey | +90 535 273 6199 | mohammedaljomaily06@gmail.com

v01d.dev | github.com/V01Ddev

Preface

I am a Computer Engineering student at Istanbul Aydın University, and this is a record of the software I have designed and shipped.

Most of it came from one job. During a gap year I took an internship at Applab in Doha that turned into fourteen months of production work for enterprise clients, and it taught me what the gap is between code that runs and code that keeps running. The platform I worked on was used by real people at scale, and the failure handling around it mattered as much as the features did.

Outside of work I build systems to understand them: a retrieval-augmented chatbot over university documentation, a PDF tool that picked up a contributor from outside, and analysis of turbine engine data because the dataset was interesting. Where a project is unfinished or was not continued, I have said so.

Al Jazeera: AI Video Platform

2025–2026

My main body of work at **Applab**, and by some distance the largest system I have worked on. The platform takes an uploaded video, strips silence and filler words, re-transcribes it, and produces a translated video with subtitles. I was the primary author of the backend: the API service, the asynchronous job worker, and the projects API.

Architecture. An event-driven pipeline on **Azure Container Apps**. A **Quart** API running under Hypercorn writes a state document to **Cosmos DB** and pushes a message onto **Azure Storage Queues**. A second, ingress-disabled container polls the queue and dispatches the work, then writes status back to Cosmos. The two services scale on different signals: the API on concurrent HTTP requests, the worker on queue depth.

Media and AI. I integrated **Azure Cognitive Services Speech** using the fast transcription endpoint, and the Video Translation API, which I migrated to a newer API version. Transcription covers **14 locales**, with automatic fallback to **Azure OpenAI Whisper** for anything outside that set. Translation and analysis run against **GPT-4o**.

The hard parts. Long-running media work fails in ways that are not obvious. I built poison-message routing so a job that will never succeed stops blocking the queue, a bounded retry loop with backoff around the translation lifecycle, and per-action state logging so a stuck job can be traced. I also wrote the subtitle alignment logic, including WebVTT handling that uses half-open interval arithmetic so adjacent cues which merely touch at a boundary do not get merged into one.

Deduplication of filler words is done through the model's structured output rather than free text: transcript word arrays are batched and passed with a schema the API is constrained to, at zero temperature, with concurrency capped by a semaphore so a long transcript cannot exhaust the rate limit.

General Authority of Customs: AI Chatbot

2025–2026

An enterprise AI chatbot built from scratch for a government client, on **FastAPI** and **Semantic Kernel**. Retrieval runs through **Azure AI Search** with vector search for semantic document lookup rather than keyword matching, and I wrote the conversation history layer myself rather than taking it from a framework, which meant owning how context is trimmed and carried between turns.

Invest Qatar: Document Generation

2026

A backend that turns AI-generated structured JSON into formatted **DOCX** documents. The team's assumption was that this needed an external service; I built it in-process with Python file I/O instead, at roughly **30 ms per document**, which removed a network dependency and a per-document cost from the pipeline.

University RAG Chatbot

2025

A retrieval-augmented chatbot answering questions over Istanbul Aydın University's documentation. **LangChain** with OpenAI embeddings and **MongoDB Atlas Vector Search** for retrieval, and a companion **Selenium** scraper that walked the university site and extracted **184 documents** from a 200-entry sitemap.

The part I would point at is the ingest pipeline. Rather than rebuilding the index on every run, it content-hashes each page and diffs against what is already stored, so only pages that actually changed are inserted, updated or deleted. The site changes constantly and most of it does not change at all; this makes re-ingesting cheap instead of an event.

TIPOT: PDF Organising Tool

2024–2026

A web application for reordering and merging PDF pages, built with **Flask** and **PyMuPDF** and shipped as a Docker image with a compose file. It is the project I am most glad I put online: an outside contributor found it, used it, and submitted a pull request that I reviewed and merged. Maintaining something other people depend on, and integrating someone else's work into it, taught me more than the code did.

Aircraft Engine Trend Analysis

2026

Analysis of multi-model turbine engine trend data for the 747, 767, 777 and 787 in Python. I worked through exhaust gas temperature, fuel flow, and N1 and N2 vibration signatures, comparing behaviour across the fleet and over time, and wrote up what the correlations suggested about wear, sensor bias and thermal drift. It started as a dataset I found interesting and turned into an exercise in not over-reading a trend line.

Smaller Projects

- **v01d.dev** — Personal site and technical blog, hand-built with Hugo and maintained since 2021, including write-ups of hardware work.
- **deltastack.dev** — Landing site for a student engineering team; Next.js, React and Tailwind. Sole author.
- **table2csv** — Converts the university course-schedule site into printable and Google Calendar CSV, with a manual-paste fallback for pages that block automation.
- **LAN Scanner** — Threaded network ping sweep, implemented in both C++ and Python.

Toolchain. Python, Go, C, TypeScript. Azure: Container Apps, Cosmos DB, Blob and Queue Storage, Cognitive Services, Azure OpenAI, AI Search. FastAPI, Quart, Semantic Kernel, LangChain. Docker, Linux. Cosmos DB, MongoDB, Postgres, pandas. Git.